

UNINTER — CENTRO UNIVERSITÁRIO INTERNACIONAL

ESCOLA SUPERIOR POLITÉCNICA

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS / ENGENHARIA DE
SOFTWARE

PROJETO MULTIDISCIPLINAR — TRILHA BACK-END (2026)

RELATÓRIO TÉCNICO E DOCUMENTAÇÃO DE SOFTWARE

REDE DE LANCHONETES "RAÍZES DO NORDESTE"

Aluno: Diego Henrique Magalhães RU: 4700827 Disciplina: Projeto Multidisciplinar — Trilha Back-End

Orientadora: Prof. Me. Luciane Yanase Kanashiro

Ano: 2026

SUMÁRIO

1. INTRODUÇÃO E OBJETIVOS

2. ANÁLISE E REQUISITOS

- 2.1 Contexto do Negócio
- 2.2 Requisitos Funcionais (RF)
- 2.3 Requisitos Não Funcionais (RNF)
- 2.4 Requisito Obrigatório de Multicanalidade (canalPedido)

3. MODELAGEM E ARQUITETURA DE BACK-END

- 3.1 Diagrama de Casos de Uso e Especificação do Fluxo Crítico
- 3.2 Modelo Entidade-Relacionamento (DER)
- 3.3 Arquitetura do Sistema (Clean Architecture / Camadas)
- 3.4 Diagrama de Classes (Visão de Domínio)

- 3.5 Estratégia de Execução e Orquestração (Docker vs .NET CLI)
 - 3.6 Diagrama de Sequência do Fluxo Crítico
 - 4. [CONTRATO DE API E ENDPOINTS](#)
 - 4.1 Padrão de Erro Consistente (JSON)
 - 4.2 Detalhamento de Todos os Endpoints por Recurso
 - 5. [LGPD, PRIVACIDADE E SEGURANÇA NO BACK-END](#)
 - 6. [PLANO DE TESTES E EVIDÊNCIAS DE EXECUÇÃO](#)
 - 7. [CONCLUSÃO](#)
 - 8. [REFERÊNCIAS](#)
-

1. INTRODUÇÃO E OBJETIVOS

A expansão da rede de lanchonetes "**Raízes do Nordeste**" exige a modernização e consolidação da sua infraestrutura tecnológica para integrar múltiplos canais de atendimento (Aplicativo Mobile, Totens Autoatendimento, Balcão Presencial, Pickup e Plataforma Web).

Este trabalho apresenta o projeto e a implementação de uma **API Back-End RESTful**, desenvolvida em C# / .NET 10 sob os preceitos de Clean Architecture e Domain-Driven Design (DDD). O sistema tem como objetivo prover alta disponibilidade, integridade transacional no gerenciamento de estoque por unidade, segurança das informações conforme a LGPD, além de garantir rastreabilidade operacional de pedidos por canal de origem.

2. ANÁLISE E REQUISITOS

2.1 Contexto do Negócio

A rede opera em formato de matriz e franquias/unidades descentralizadas. Cada unidade fabril/comercial gerencia seu próprio cardápio, estoque de insumos e fila de produção na cozinha.

2.2 Requisitos Funcionais (RF)

- **RF01 — Autenticação e Perfis:** Permitir cadastro e login com emissão de tokens JWT e controle de acesso por roles (`Customer` , `Professional` , `Manager` , `Admin` , `Owner`), além de renovação por Refresh Token e logout.
- **RF02 — Gestão de Unidades:** Permitir a consulta e listagem das unidades ativas da rede.
- **RF03 — Cardápio e Produtos por Unidade:** Permitir a criação de cardápios, cadastro, atualização, remoção e consulta de produtos e preços vigentes por unidade.
- **RF04 — Realização de Pedidos:** Permitir a criação de pedidos associando cliente, unidade, itens, quantidades e canal de origem (`canalPedido`).

- **RF05 — Atualização de Status do Pedido:** Permitir a transição de estados do pedido (`Pendente` → `EmPreparacao` → `Pronto` → `Entregue` / `Cancelado`).
- **RF06 — Controle de Estoque por Unidade:** Registrar a criação de estoques e movimentações (entrada, saída, perda, consumo) de ingredientes, impedindo vendas caso o estoque seja insuficiente.
- **RF07 — Programa de Fidelidade:** Permitir a adesão (`POST /loyalty`) e o cancelamento (`DELETE /loyalty`) de participação no programa de fidelização mediante consentimento explícito do cliente.
- **RF08 — Integração de Pagamento Mock & Worker:** Processar simulação de pagamento via gateway externo, enfileiramento via worker background (`UninterPayment.Worker`) e recepção de notificações via webhook assíncrono.

2.3 Requisitos Não Funcionais (RNF)

- **RNF01 — Segurança:** Criptografia de senhas utilizando BCrypt (cost factor 11) e autenticação Stateless via JWT + Refresh Tokens.
- **RNF02 — LGPD e Privacidade:** Minimização de dados pessoais expostos nos endpoints de resposta e registro do consentimento do cliente.
- **RNF03 — Padrão REST e OpenAPI:** Endpoints estruturados com URLs no plural/singular padronizados, status codes apropriados e especificação OpenAPI/Scalar Reference.
- **RNF04 — Persistência e Integridade:** ORM Entity Framework Core com suporte a Migrations e chave estrangeira em cascata.

2.4 Requisito Obrigatório de Multicanalidade (`canalPedido`)

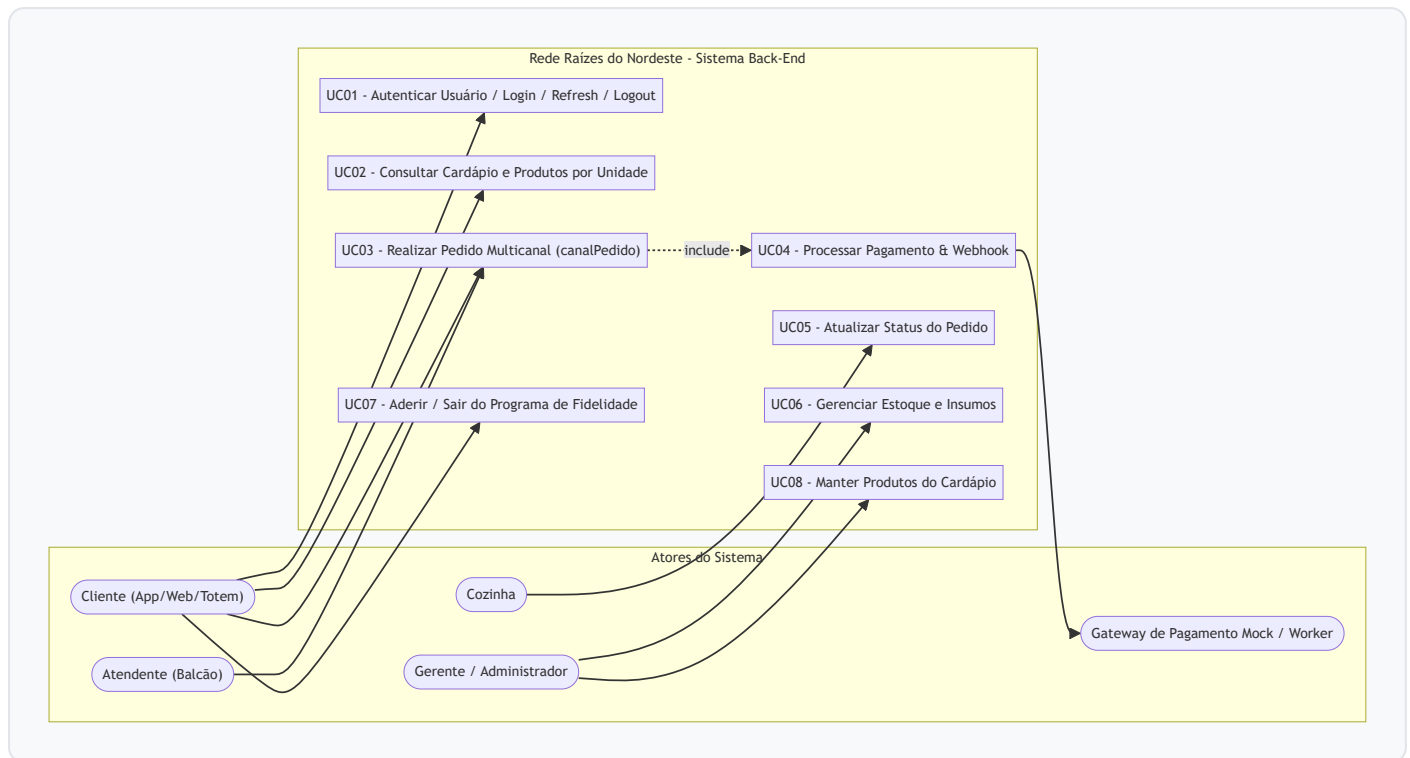
Para garantir a rastreabilidade operacional dos múltiplos canais de vendas da rede, todo pedido possui obrigatoriamente o atributo `channel` (mapeado na API como `canalPedido`), estruturado como um `ENUM` :

- `APP` (0)
- `TOTEM` (1)
- `BALCAO` (2)
- `PICKUP` (3)
- `WEB` (4)

A criação do pedido rejeita chamadas que omitam o `canalPedido` , e a API permite filtragem de relatórios por canal (`GET /pedido?canalPedido=1`).

3. MODELAGEM E ARQUITETURA DE BACK-END

3.1 Diagrama de Casos de Uso

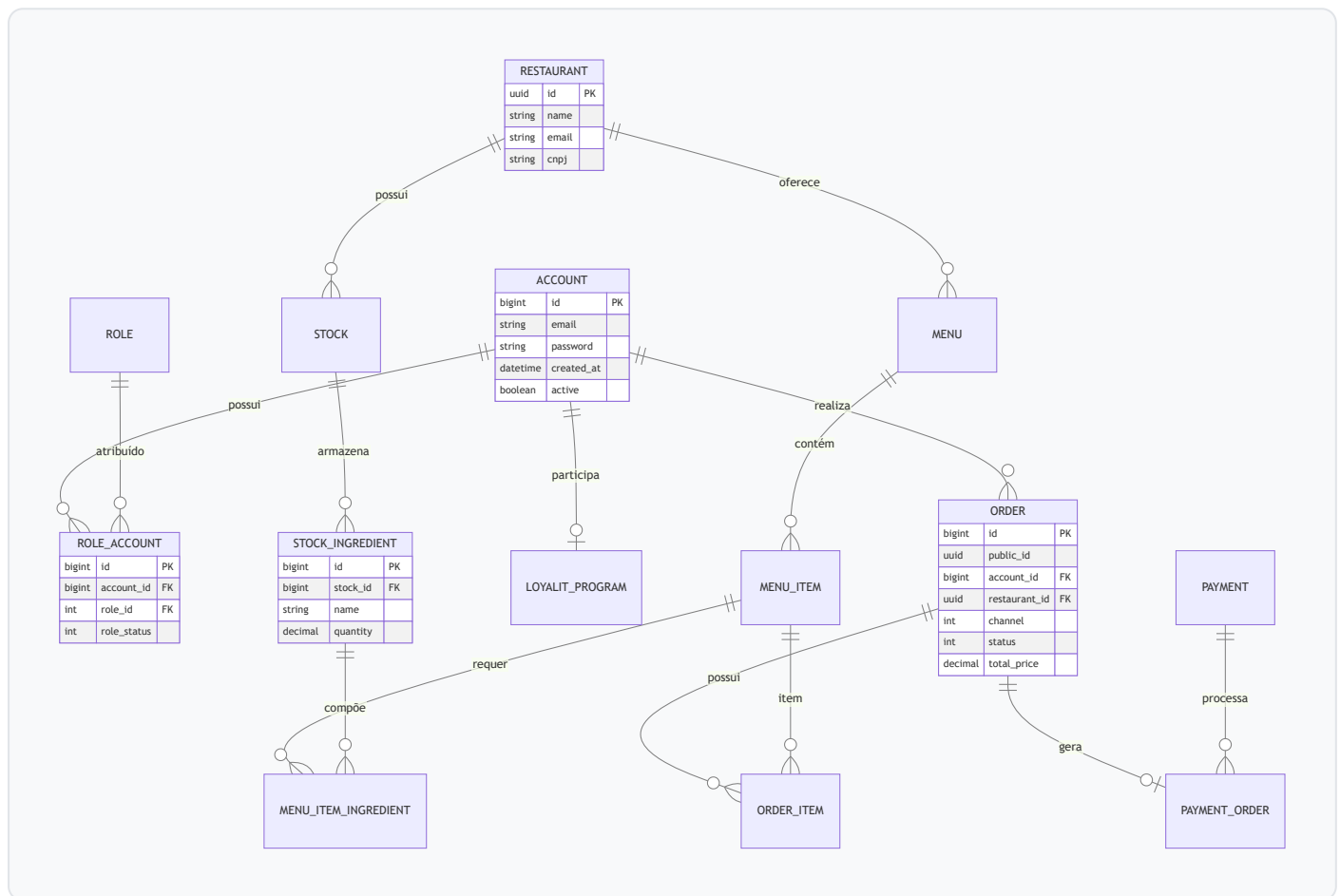


Especificação Detalhada do Caso de Uso Crítico (UC03 — Realizar Pedido + Pagamento Mock)

- **Ator Principal:** Cliente (App/Web/Totem) / Atendente (Balcão).
- **Demais Atores:** Cozinha, Gateway de Pagamento.
- **Pré-condições:** Usuário autenticado, unidade ativa selecionada e itens disponíveis em estoque.
- **Fluxo Principal:**
 - i. O cliente escolhe a unidade e os produtos informando a quantidade e o `canalPedido` (ex: `1` para `TOTEM`).
 - ii. O sistema verifica a disponibilidade de cada insumo no estoque da unidade.
 - iii. O sistema reserva os insumos, cria o pedido com status `Pending` (Pendente) e calcula o valor total.
 - iv. O cliente solicita a emissão do pagamento (`POST /pagamento/pedido/{orderId}`).
 - v. Após processamento/webhook do gateway mock, o status transita para `EmPreparacao` (Cozinha) e efetiva a baixa dos insumos no estoque.
- **Pós-condições:**
 - Pedido gravado no banco de dados com status `Pending` (ou `EmPreparacao` pós-pagamento) e identificador único `publicId` (GUID).
 - Baixa e reserva de estoque dos insumos correspondentes efetuada no banco de dados da unidade.

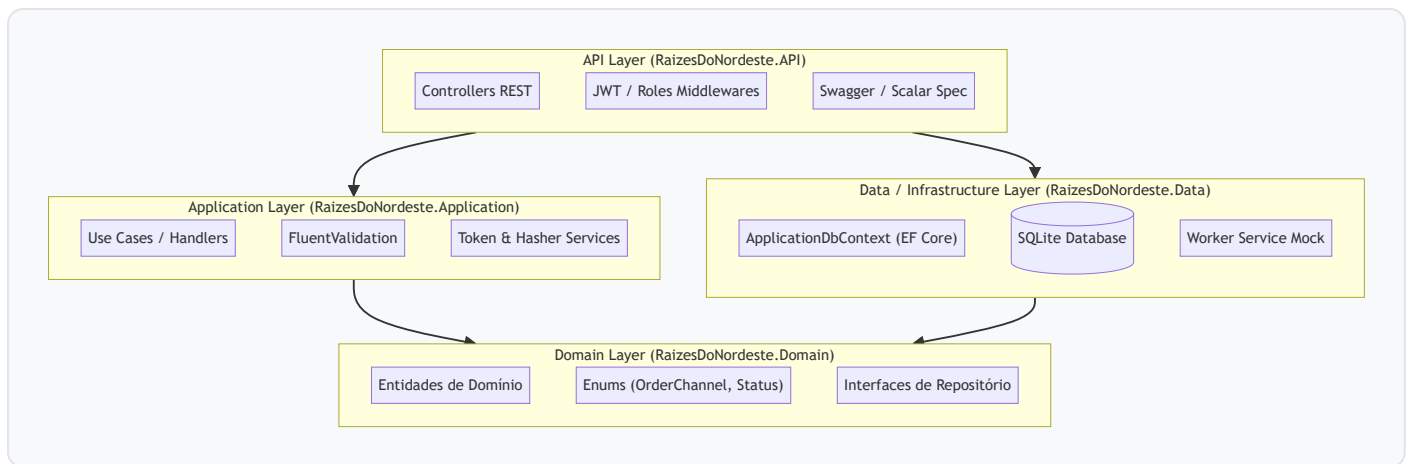
- Rastreabilidade de multicanalidade (`canalPedido`) registrada para consulta e relatórios.
- Notificação emitida para a fila de produção da Cozinha.
- **Exceções:**
 - *Estoque Insuficiente*: Retorna erro `HTTP 409 Conflict` detalhando o item indisponível.
 - *Canal Inválido/Ausente*: Retorna erro `HTTP 400 Bad Request`.
 - *Pagamento Recusado*: Atualiza o status do pedido para `Cancelled` (Cancelado) e estorna a reserva do estoque.

3.2 Modelo Entidade-Relacionamento (DER)



3.3 Arquitetura do Sistema (Clean Architecture / Camadas)

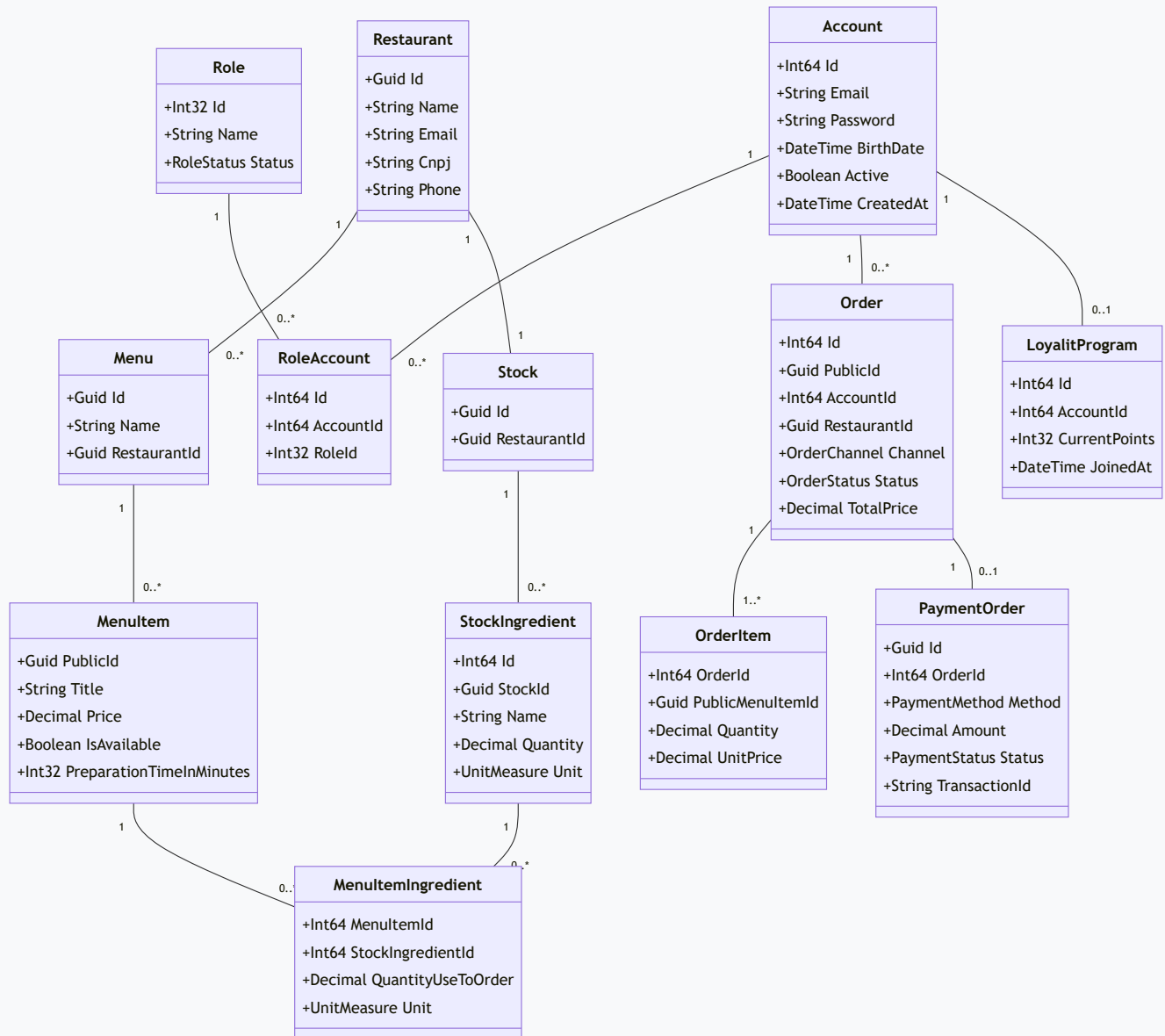
A solução foi estruturada em 4 projetos distintos respeitando o princípio da inversão de dependência (Clean Architecture / DDD):



1. **RaizesDoNordeste.Domain** : Camada core contendo Entidades de Domínio, Objetos de Valor (`Email`), Enums (`OrderChannel` , `OrderStatus`), Interfaces de Repositórios e Regras de Negócio fundamentais.
2. **RaizesDoNordeste.Application** : Casos de Uso (`UseCases` e `Dispatchers`), Validações (`FluentValidation`), Serviços de Criptografia (`HasherService`), Geração de Tokens (`TokenService`) e Mapeamentos.
3. **RaizesDoNordeste.Data** : Infraestrutura de dados, mapeamentos EF Core (`EntityBuilders`), Configurações do Banco de Dados SQLite e Migrations.
4. **RaizesDoNordeste.API** : Controllers REST, Middlewares de Autenticação JWT, Filtros de Autorização por Role (`[RolesAuthorize]`) e Documentação Swagger/Scalar Reference.

3.4 Diagrama de Classes (Visão de Domínio)

O diagrama de classes a seguir apresenta o modelo de domínio do sistema, detalhando as entidades principais, seus atributos e relacionamentos:



3.5 Estratégia de Execução e Orquestração (Docker vs .NET CLI)

A aplicação foi projetada para suportar duas modalidades de execução:

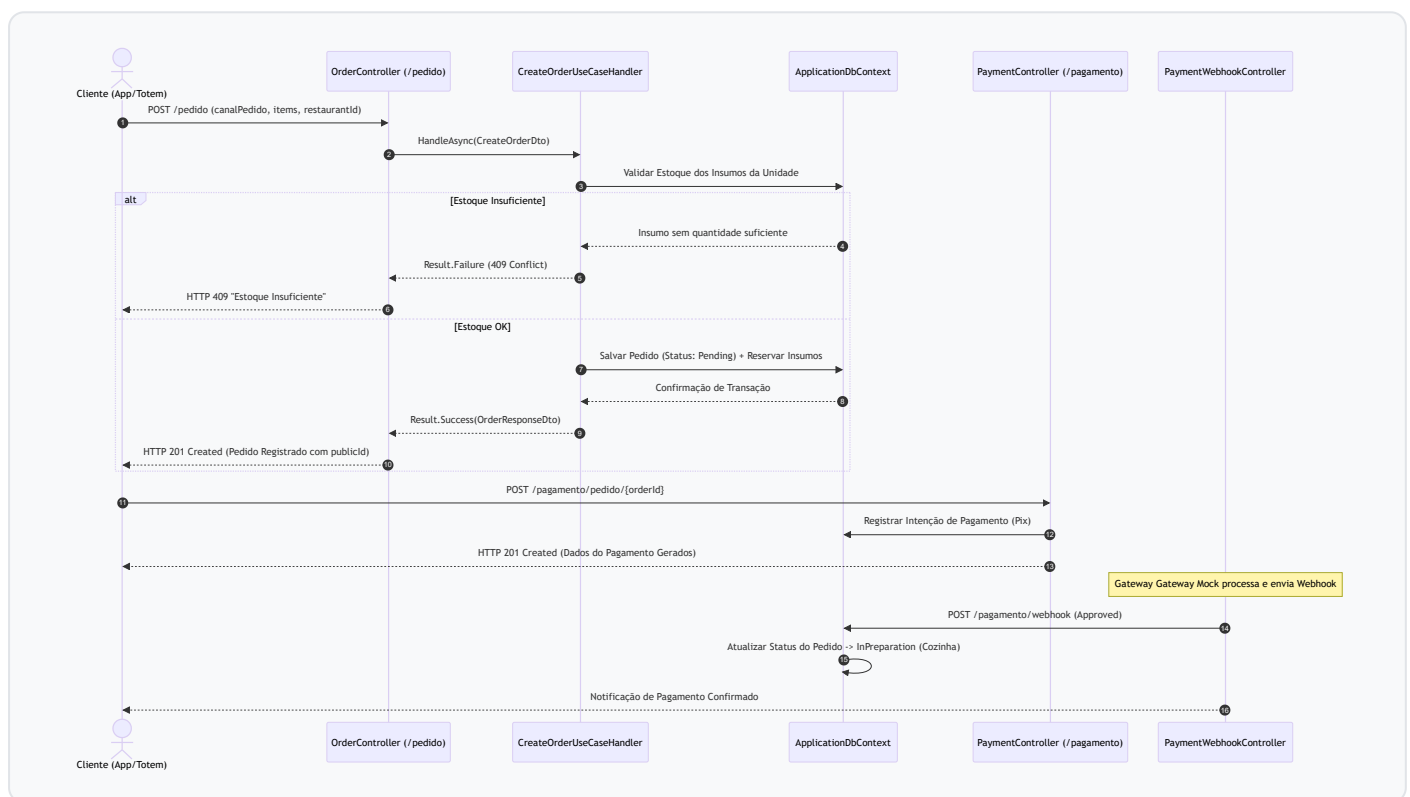
1. Orquestração via Docker Compose (API + Worker Simultâneos):

- **Dockerfile.api** : Container multi-stage compilando e executando a API principal na porta fixa `5269` (expondo também o mapeamento `8080`).
- **Dockerfile.worker** : Container multi-stage executando o serviço em segundo plano `UninterPayment.Worker` na porta fixa `5200`.
- **docker-compose.yml** : Cria a rede isolada `raizes-network` conectando a API ao Worker via `http://worker:5200`.
- **Comando**: `docker-compose up --build`

2. Execução Individual via .NET CLI / IIS Express (`dotnet run`):

- **API Principal:** `dotnet run --project src/RaizesDoNordeste.API/RaizesDoNordeste.API.csproj` (escuta na mesma porta fixa `http://localhost:5269` e `https://localhost:7081`).
- **Payment Worker:** `dotnet run --project src/UninterPayment.Worker/UninterPayment.Worker.csproj` (escuta na porta fixa `http://localhost:5200`).

3.6 Diagrama de Sequência — Fluxo Crítico de Pedido e Pagamento



4. CONTRATO DE API E ENDPOINTS

4.1 Padrão de Erro Consistente (JSON)

Todas as falhas da API seguem rigorosamente a estrutura padronizada de erro abaixo:

```
{
  "code": 409,
  "message": "Falha ao criar o pedido",
  "validations": [
```



```

{
  "property": "Items[0].Quantity",
  "errors": [
    "Estoque insuficiente para o produto especificado."
  ]
}
]
}

```

4.2 Detalhamento de Todos os Endpoints por Recurso

A API expõe 23 endpoints divididos entre a API principal e o serviço worker de pagamentos:

4.2.1 Autenticação (/auth)

1. POST /auth/login

- **Permissão:** Público
- **Descrição:** Autentica o usuário com e-mail, senha e ID da unidade, retornando o token JWT e o Refresh Token.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
email	string	Sim	E-mail do usuário cadastrado (admin@raizesdonordeste.com)
password	string	Sim	Senha de acesso do usuário (somehashedpassword)
restaurantId	Guid	Sim	Identificador da unidade ativa (9a88024d-2618-4e25-87f5-35217f7a7c8a)

- **Status HTTP:** 200 OK , 400 Bad Request , 401 Unauthorized .

2. POST /auth/refresh

- **Permissão:** Público
- **Descrição:** Renova o Access Token expirado a partir de um Refresh Token válido.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
refreshToken	string	Sim	Token de renovação ativo (eyJhbGciOi...)

- **Status HTTP:** 200 OK , 400 Bad Request , 401 Unauthorized .

3. POST /auth/logout

- **Permissão:** Autenticado

- **Descrição:** Invalida o Refresh Token ativo, encerrando a sessão.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
refreshToken	string	Sim	Token de renovação a ser revogado (eyJhbGciOi...

- **Status HTTP:** 200 OK , 400 Bad Request .

4. GET /auth/desenvolvedor

- **Permissão:** Público (*Ambiente de Desenvolvimento / Testes*)
- **Descrição:** Gera automaticamente um token JWT com perfil Administrador para a unidade Matriz.
- **Status HTTP:** 200 OK , 404 Not Found (em Produção) .

4.2.2 Usuários (/usuarios)

5. POST /usuarios

- **Permissão:** Público
- **Descrição:** Cadastra um novo usuário no sistema (recebe perfil padrão Customer). Exige a data de nascimento (birthDate).
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
email	string	Sim	E-mail válido do cliente (novocliente@raizesdonordeste.com)
password	string	Sim	Senha (mínimo 6 caracteres, letras e números) (Senha@123456)
birthDate	DateTime	Sim	Data de nascimento no formato ISO-8601 (1995-05-15T00:00:00Z)

- **Status HTTP:** 201 Created , 400 Bad Request .

6. GET /usuarios/perfil

- **Permissão:** Autenticado
- **Descrição:** Retorna as informações cadastrais e perfis associados à conta logada.
- **Status HTTP:** 200 OK , 401 Unauthorized .

4.2.3 Unidades e Restaurantes (/unidades)

7. GET /unidades

- **Permissão:** JWT Autenticado (Admin , Owner)

- **Descrição:** Lista todos os restaurantes e unidades ativas da rede. Suporta paginação via query string (`?page=1&limit=10`).
- **Status HTTP:** `200 OK` , `401 Unauthorized` , `403 Forbidden` .

8. **POST** /unidades

- **Permissão:** JWT Autenticado (`Admin` , `Owner`)
- **Descrição:** Cadastra uma nova franquia/unidade da rede (com criação automática do container de estoque).
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
<code>name</code>	<code>string</code>	Sim	Nome da unidade (<code>Raízes do Nordeste - Unidade Recife</code>)
<code>description</code>	<code>string</code>	Sim	Descrição da unidade (<code>Unidade de atendimento no centro de Recife</code>)
<code>phone</code>	<code>string</code>	Sim	Telefone de contato (<code>81988887777</code>)
<code>email</code>	<code>string</code>	Sim	E-mail corporativo único (<code>recife@raizesdonordeste.com</code>)
<code>cnpj</code>	<code>string</code>	Sim	CNPJ válido de 14 dígitos (<code>12345678000276</code>)
<code>addressStreet</code>	<code>string</code>	Sim	Logradouro / Rua (<code>Rua do Sol</code>)
<code>addressNumber</code>	<code>string</code>	Sim	Número do imóvel (<code>100</code>)
<code>addressDistrict</code>	<code>string</code>	Sim	Bairro (<code>Santo Antônio</code>)
<code>addressCity</code>	<code>string</code>	Sim	Cidade (<code>Recife</code>)
<code>addressState</code>	<code>string</code>	Sim	Sigla do Estado UF (<code>PE</code>)
<code>addressZipCode</code>	<code>string</code>	Sim	CEP de 8 dígitos (<code>50010000</code>)

- **Status HTTP:** `201 Created` , `400 Bad Request` , `403 Forbidden` , `409 Conflict` .

4.2.4 Cardápio / Menu (/cardapio)

9. **POST** /cardapio

- **Permissão:** JWT Autenticado (`Manager` , `Owner` , `Admin`)
- **Descrição:** Cria uma nova estrutura de cardápio (menu) vinculada a uma unidade.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
<code>name</code>	<code>string</code>	Sim	Nome do cardápio (<code>Cardápio Principal</code>)
<code>restaurantId</code>	<code>Guid</code>	Sim	Identificador da unidade (<code>9a88024d-2618-4e25-87f5-35217f7a7c8a</code>)

- **Status HTTP:** 201 Created , 400 Bad Request , 403 Forbidden .

10. GET /cardapio/usuario-atual

- **Permissão:** Autenticado
- **Descrição:** Obtém o cardápio e os itens cadastrados para a unidade do usuário logado.
- **Status HTTP:** 200 OK , 401 Unauthorized .

4.2.5 Itens do Cardápio (/cardapio/itens ou /menu/itens)

11. POST /cardapio/itens

- **Permissão:** JWT Autenticado (Manager , Owner , Admin)
- **Descrição:** Cadastra um novo item no cardápio da unidade com a lista de ingredientes. Permite vincular insumos existentes ou cadastrar novos ingredientes no estoque.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
title	string	Sim	Título do item (Tapioca de Carne de Sol)
description	string	Não	Descrição detalhada do prato/bebida
price	decimal	Sim	Preço de venda em R\$ (18.50)
imageUrl	string	Não	Link da imagem do produto
isAvailable	boolean	Sim	Indica se está disponível para venda (true)
displayOrder	int	Sim	Ordem de exibição no menu (1)
preparationTimeInMinutes	int	Sim	Tempo estimado de preparo em minutos (15)
isFeatured	boolean	Sim	Se é destaque do cardápio (true)
ingredients	Array	Não	Lista de insumos vinculados do estoque
ingredients[].stockIngredientId	long	Opcional	ID do insumo existente no estoque (1)
ingredients[].name	string	Opcional	Nome do novo ingrediente a cadastrar
ingredients[].unit	enum	Sim	Unidade medida (0 =Gram, 1 =ML, 2 =Unit, 3 =Kg, 4 =L)
ingredients[].quantityUseToOrder	decimal	Sim	Quantidade consumida por porção do pedido (0.250)
ingredients[].initialStockQuantity	decimal	Opcional	Quantidade inicial de estoque caso seja um novo insumo (50.0)

- **Status HTTP**: `201 Created`, `400 Bad Request`, `403 Forbidden`.

12. GET /cardapio/itens

- **Permissão:** Autenticado
- **Descrição:** Lista todos os itens do cardápio cadastrados na unidade corrente. Suporta paginação via query string (`?page=1&limit=10`).
- **Status HTTP:** `200 OK` , `401 Unauthorized` .

13. `GET /cardapio/itens/{id}`

- **Permissão:** Autenticado
- **Descrição:** Obtém os detalhes de um item do cardápio específico pelo seu ID numérico ou PublicId GUID.
- **Status HTTP:** `200 OK` , `404 Not Found` .

14. `PUT /cardapio/itens/{id}`

- **Permissão:** JWT Autenticado (`Manager` , `Owner` , `Admin`)
- **Descrição:** Atualiza os dados cadastrais de um item do cardápio existente.
- **Status HTTP:** `200 OK` , `400 Bad Request` , `404 Not Found` .

15. `DELETE /cardapio/itens/{id}`

- **Permissão:** JWT Autenticado (`Manager` , `Owner` , `Admin`)
- **Descrição:** Remove um item do cardápio.
- **Status HTTP:** `200 OK` , `404 Not Found` .

16. `POST /cardapio/itens/{menuItemId}/ingredientes`

- **Permissão:** JWT Autenticado (`Manager` , `Owner` , `Admin`)
- **Descrição:** Vincula um ingrediente de estoque a um item do cardápio. Aceita insumo existente por ID/PublicId ou cadastro de novo insumo por nome.
- **Mapeamento de Atributos (Request Body):**

Cenário de Uso	Campo	Tipo	Requerido	Descrição / Exemplo
Insumo Existente	<code>publicStockIngredientId</code>	<code>Guid</code>	Opcional	Public ID do insumo (<code>11111111-1111...</code>)
	<code>stockIngredientId</code>	<code>long</code>	Opcional	ID numérico do insumo (<code>1</code>)
	<code>quantityUseToOrder</code>	<code>decimal</code>	Sim	Consumo por pedido (<code>0.250</code>)
Novo Insumo	<code>name</code>	<code>string</code>	Sim	Nome do novo insumo (<code>Manteiga de Garrafa</code>)
	<code>unit</code>	<code>enum</code>	Sim	Unidade de medida (<code>1</code> = Milliliter)
	<code>quantityUseToOrder</code>	<code>decimal</code>	Sim	Consumo por pedido (<code>0.050</code>)
	<code>initialStockQuantity</code>	<code>decimal</code>	Sim	Quantidade inicial no estoque (<code>10.0</code>)

- **Status HTTP**: `201 Created`, `400 Bad Request`, `404 Not Found`.

4.2.6 Controle de Estoque (/estoque)

17. POST /estoque/ingredientes

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Cadastra um novo insumo bruto/ingrediente direto no estoque da unidade logada.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
name	string	Sim	Nome do ingrediente (Farinha de Mandioca Nova)
unit	enum	Sim	Unidade de medida (0 =Gram, 1 =Ml, 2 =Unit, 3 =Kg, 4 =L)
quantity	decimal	Sim	Quantidade de estoque inicial (50.0)

- **Status HTTP**: `201 Created`, `400 Bad Request`.

18. POST /estoque/movimentacao

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Registra movimentação manual de estoque (Entry =0/Abastecimento, Consumption =1/Saída, Loss =2/Desperdício, Adjustment =3).
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
publicStockIngredientId	Guid	Opcional	Public ID do insumo (11111111-1111...)
stockIngredientId	long	Opcional	ID numérico do insumo (1)
quantity	decimal	Sim	Quantidade a movimentar (50.0)
type	enum	Sim	Tipo (0 =Entry, 1 =Consumption, 2 =Loss, 3 =Adjustment)
description	string	Não	Observação da movimentação (Entrada de fornecedor)

- **Status HTTP**: `200 OK`, `400 Bad Request`.

19. GET /estoque

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Consulta a quantidade atual de insumos no estoque da unidade logada.

- **Status HTTP:** 200 OK , 401 Unauthorized .

20. GET /estoque/unidade/{restaurantId}

- **Permissão:** JWT Autenticado (Manager , Owner , Admin)
- **Descrição:** Consulta o estoque de uma unidade específica da rede informando seu GUID.
- **Status HTTP:** 200 OK , 404 Not Found .

4.2.7 Gestão de Pedidos (/pedido)

21. POST /pedido

- **Permissão:** Autenticado
- **Descrição:** Abertura de pedido com validação e dedução de estoque dos insumos. Exige o canalPedido .
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
canalPedido	enum	Sim	Canal de venda (0 =APP, 1 =TOTEM, 2 =BALCAO, 3 =PICKUP, 4 =WEB)
items	Array	Sim	Lista de itens solicitados no pedido
items[].publicMenuItemId	Guid	Sim	GUID do item no cardápio (9a88024d-2618-4e25-87f5-35217f7a7c9b)
items[].quantity	decimal	Sim	Quantidade de porções (1)

- **Status HTTP**: `201 Created`, `400 Bad Request`, `409 Conflict`.

22. GET /pedido

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Lista os pedidos da unidade com suporte a filtros query string e paginação (? canalPedido=1&status=1&page=1&limit=10).
- **Status HTTP:** 200 OK , 403 Forbidden .

23. GET /pedido/{id}

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Consulta o detalhamento completo de um pedido pelo seu publicId (GUID).
- **Status HTTP:** 200 OK , 404 Not Found .

24. PUT /pedido/status/{id}

- **Permissão:** JWT Autenticado (Professional , Manager , Owner , Admin)
- **Descrição:** Altera o status de processamento do pedido.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
status	enum	Sim	Novo status (0 =Pending, 1 =InPreparation, 2 =Ready, 3 =Delivered, 4 =Canceled)

- **Status HTTP**: `200 OK`, `400 Bad Request`.

4.2.8 Processamento de Pagamento (/pagamento)

25. POST /pagamento/pedido/{orderId}

- **Permissão:** Autenticado
- **Descrição:** Dispara o pedido de cobrança/processamento de pagamento para o pedido especificado.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
paymentMethod.method	enum	Sim	Método (0 = Pix, 1 = Cartão de Crédito)
loyaltyPointToUse	int	Opcional	Quantidade de pontos de fidelidade a resgatar como desconto (0)

- **Status HTTP**: `201 Created`, `400 Bad Request`.

26. POST /pagamento/webhook

- **Permissão:** Autenticado via Header X-Uninter-Signature
- **Descrição:** Webhook receptor de confirmações assíncronas do Gateway de Pagamento Mock.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
transactionId	string	Sim	Identificador único da transação no gateway (trans_12345abcde)
orderId	Guid	Sim	Public ID do pedido no sistema (3fa85f64-5717-4562-b3fc-2c963f66afa6)
status	string	Sim	Status da cobrança ("Approved")
amount	decimal	Sim	Valor total do pagamento (29.90)

- **Status HTTP**: `200 OK`, `400 Bad Request`, `404 Not Found`.

4.2.9 Programa de Fidelidade (/loyalty)

27. POST /loyalty

- **Permissão:** JWT Autenticado (Manager , Owner , Admin)
- **Descrição:** Registra o aceite do cliente e sua entrada no programa de fidelidade.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
customerAccountId / accountId	long	Sim	ID da conta de usuário do cliente (1)

- **Status HTTP**: `201 Created`, `400 Bad Request`.

28. DELETE /loyalty

- **Permissão:** Autenticado
- **Descrição:** Cancela a adesão do cliente ao programa de fidelidade.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
customerAccountId / accountId	long	Opcional	ID da conta do cliente (opcional se logado)

- **Status HTTP**: `200 OK`, `400 Bad Request`.

4.2.10 Worker Mock de Pagamentos (UninterPayment.Worker — Porta 5200)

29. POST http://localhost:5200/payments

- **Permissão:** Livre / Interno
- **Descrição:** Recebe requisições de pagamento na fila em memória do serviço Worker.
- **Mapeamento de Atributos (Request Body):**

Campo / Propriedade	Tipo	Obrigatório	Descrição / Exemplo
transactionId	string	Sim	ID da transação (trans_12345abcde)
orderId	Guid	Sim	Public ID do pedido (3fa85f64-5717-4562-b3fc-2c963f66afa6)
amount	decimal	Sim	Valor do pagamento (29.90)

- **Status HTTP**: `202 Accepted`, `400 Bad Request`.

29. **GET** `http://localhost:5200/health`

- **Permissão**: Livre
- **Descrição**: Endpoint de healthcheck do serviço worker em segundo plano.
- **Status**: `200 OK`.

5. LGPD, PRIVACIDADE E SEGURANÇA NO BACK-END

A aplicação adota medidas técnicas e organizacionais rigorosas para conformidade com a LGPD (Lei Geral de Proteção de Dados - Lei nº 13.709/2018):

1. **Minimização de Dados (Art. 6º, III)**: Coleta-se estritamente o e-mail e a credencial de acesso necessários para a prestação do serviço.
2. **Armazenamento Seguro de Senhas**: Senhas nunca são salvas em texto plano. É utilizado o algoritmo de hashing **BCrypt** com fator de custo 11 e salting automático.
3. **Gerenciamento de Sessões e Expiração**: A autenticação utiliza tokens JWT assinados com chave simétrica SHA-256 e tempo de expiração curto, aliado a Refresh Tokens revogáveis em banco.
4. **Consentimento no Programa de Fidelidade**: A inclusão e saída no programa de fidelização exige chamadas explícitas (`POST /loyalty` e `DELETE /loyalty`), registrando a ciência do aceite.
5. **Auditoria e Logs de Ações Sensíveis**: Alterações de status de pedidos e movimentações financeiras gravam rastros de auditoria com timestamp UTC e identificação da conta responsável.

6. PLANO DE TESTES E EVIDÊNCIAS DE EXECUÇÃO

A API foi validada por meio de suítes de testes unitários automatizados em xUnit (`dotnet test`) e executada funcionalmente através da coleção completa de testes do Postman (`RaizesDoNordeste.postman_collection.json`).

Tabela Consolidada dos Cenários de Testes (10 Cenários Principais)

ID	Cenário de Teste	Endpoint / Método	Pré-condição	Entrada (Request)	Resultado Esperado	Evidência na Coleção Postman
T01	Autenticação Válida de Cliente	POST /auth/login	Usuário pre-cadastrado no banco	E-mail e senha corretos	200 OK + JWT Token no Body	Autenticação / Login Manual (Administrador)
T02	Autenticação com Senha Incorreta	POST /auth/login	Usuário existente	E-mail correto e senha errada	401 Unauthorized + Mensagem legível	Autenticação / Login - Credenciais Inválidas (Erro 401)
T03	Consulta de Unidades sem Autenticação	GET /unidades	Nenhuma	Header Authorization ausente	401 Unauthorized	Unidades e Cardápio / Listar Unidades - Acesso Sem Token
T04	Listagem de Unidades da Rede	GET /unidades	Usuário autenticado (Admin)	Token JWT no Header	200 OK + Lista de Restaurantes	Unidades e Cardápio / Listar Unidades (Restaurantes)
T05	Criação de Pedido Válido (TOTEM)	POST /pedido	Unidade e produto válidos	channel: 1 (TOTEM), items	201 Created + JSON do Pedido	Pedidos / Criar Pedido com Sucesso (Canal Válido TOTEM)
T06	Pedido com Canal de Origem Inválido	POST /pedido	Usuário autenticado	Campo channel: 99 (inválido)	400 Bad Request + Erro de Validação	Pedidos / Criar Pedido - canalPedido Inválido (Erro 400)
T07	Pedido com Estoque Insuficiente	POST /pedido	Produto com estoque baixo	quantity: 99999	409 Conflict + Erro de Estoque	Pedidos / Criar Pedido - Sem Estoque (Erro 409)
T08	Filtragem de Pedidos por Canal	GET /pedido?canalPedido=1	Perfil Admin / Professional	Query Param canalPedido=1	200 OK + Apenas pedidos do Totem	Pedidos / Listar Pedidos - Filtrado por canalPedido

ID	Cenário de Teste	Endpoint / Método	Pré-condição	Entrada (Request)	Resultado Esperado	Evidência na Coleção Postman
T09	Acesso Proibido a Cliente em Rota Gerencial	GET /pedido	Perfil Customer	Token de Cliente comum	403 Forbidden	Pedidos / Listar Pedidos - Acesso Negado Cliente (Erro 403)
T10	Renovação de Token via Refresh Token	POST /auth/refresh	Refresh token ativo no banco	refreshToken válido	200 OK + Novo JWT + Refresh Token	Autenticação / Renovar Token (Refresh Token)

7. CONCLUSÃO

A solução apresentada para a rede "**Raízes do Nordeste**" cumpriu integralmente todos os requisitos funcionais, não funcionais e arquiteturais especificados no roteiro da disciplina.

A implementação comprova maturidade técnica ao entregar:

- Arquitetura limpa e desacoplada em 4 camadas (.NET 10);
- Cobertura completa do fluxo crítico (Pedido → Validação de Estoque → Pagamento Mock / Worker → Transição de Status);
- Atendimento estrito ao requisito obrigatório de **Multicanalidade** via `canalPedido` (ENUM);
- Mecanismos robustos de segurança e LGPD (BCrypt, JWT Bearer, Refresh Tokens, autorização RBAC);
- Coleção completa no Postman cobrindo todos os 23 endpoints da solução e testes unitários automatizados.

A API se posiciona como um produto de software robusto, escalável e pronto para sustentar a operação de mercado da empresa em múltiplos canais de atendimento.